

GUÍA PRÁCTICA GRUPAL SEMANA 10

Unidad II: Desarrollo Web Fullstack

Tema: Lenguaje Python: Sintaxis, Estructuras de Control, POO (Clases, Herencia, Polimorfismo, Encapsulamiento), Excepciones y Módulos

Modalidad: Laboratorio

APELLIDOS Y NOMBRES:

- Alarcón Mendoza Estiven Rodrigo
 - Calderón Leiva Anna Olenka
 - Cruz Cruz Alexander Jhon
 - Espíritu Diaz Olayne Guadalupe Maria Isabel
 - Llanos Loznao Ricardo Alexander
 - Martínez Casas Cristhian Emilio
-

Repositorio de Github:

https://github.com/AnnaOlenka/repo_semana10

Informe

1. Arquitectura POO: Clase base Producto, clases derivadas Electronico y Libro. Herencia múltiple opcional (DescuentoMixin).

Clase base: Producto

Producto es la clase raíz de la jerarquía. Define los atributos comunes a todo tipo de producto (nombre, precio, stock) y los métodos base (calcular_precio_final, __str__, __del__). Ninguna otra clase hereda de ella directamente excepto Electronico y Libro.

```
class Producto:
    def __init__(self, nombre, precio, stock):
        self.nombre = nombre
        self.precio = precio
        self.stock = stock
```

Clase derivada: Electronico

Hereda de DescuentoMixin y Producto (herencia múltiple). Agrega el atributo garantia (meses). Llama a super().__init__() para delegar la inicialización de los atributos comunes a Producto, siguiendo el principio DRY.

```
class Electronico(DescuentoMixin, Producto):
    def __init__(self, nombre, precio, stock, garantia):
        super().__init__(nombre, precio, stock)
        self.garantia = garantia
```

Clase derivada: Libro

Hereda solo de Producto (herencia simple). Agrega el atributo autor. También usa `super().__init__()` para reutilizar la inicialización base.

```
class Libro(Producto):
    def __init__(self, nombre, precio, stock, autor):
        super().__init__(nombre, precio, stock)
        self.autor = autor
```

Herencia múltiple opcional: DescuentoMixin

Es un Mixin: una clase auxiliar que no representa una entidad del dominio, sino que aporta un comportamiento reutilizable (`aplicar_descuento`). `Electronico` lo hereda para poder calcular su precio con descuento. El orden (`DescuentoMixin, Producto`) es intencional para que el MRO (Method Resolution Order) de Python resuelva correctamente.

```
class DescuentoMixin:
    def aplicar_descuento(
        self,
        precio,
        porcentaje):
        return precio * (
            1 - porcentaje / 100
        )
```

2. Encapsulamiento: Atributos privados/protegidos, acceso vía `@property`, validación de rangos en setters.

Atributos protegidos con `@property`

Los atributos reales se almacenan con prefijo `_` (`_nombre`, `_precio`, `_stock`), lo que indica por convenio que son protegidos (no deben accederse directamente desde fuera). El acceso externo se hace siempre a través de la `@property`, que actúa como getter limpio.

```

@property
def nombre(self):
    return self._nombre

@nombre.setter
def nombre(self, valor):
    if not valor.strip():
        raise ValueError(
            "Nombre inválido"
        )
    self._nombre = valor

@property
def precio(self):
    return self._precio

@precio.setter
def precio(self, valor):
    validar_precio(valor)
    self._precio = valor

@property
def stock(self):
    return self._stock

```

Validación en setters

Los setters delegan la validación a funciones del módulo `utils/validators.py`. Esto garantiza que nunca se pueda asignar un precio negativo o stock inválido, ni desde el constructor ni desde cualquier otro punto del código.

```

@nombre.setter
def nombre(self, valor):
    if not valor.strip():
        raise ValueError(
            "Nombre inválido"
        )
    self._nombre = valor

```

```

@precio.setter
def precio(self, valor):
    validar_precio(valor)
    self._precio = valor

```

3. Polimorfismo: Método `calcular_precio_final()` sobrescrito por tipo. `__str__` para representación uniforme.

Al reemplazar el método `__str__`, propio de las clases en Python, estamos haciendo que el llamado de los métodos dentro de un método como “print”, que nos permite imprimir la información por pantalla, se realice de una manera personalizada.

- Ejemplo en la clase `Producto`

```
models > Producto.py > ...
6 class Producto:
43     def calcular_precio_final(self):
44         return self.precio
45
46     def __str__(self):
47         return (
48             f"{self.nombre} | "
49             f"$ / {self.calcular_precio_final():.2f}"
50         )
51
```

- Ejemplo en la clase `Libro`

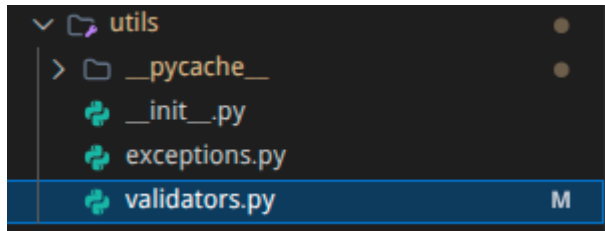
```
models > Libro.py > Libro
3 class Libro(
28 |
29     def calcular_precio_final(self):
30         return self.precio * 1.04
31 |
32     def __str__(self):
33         return (
34             f"[Libro] {self.nombre} | "
35             f"$ / {self.calcular_precio_final():.2f} "
36             f"(Autor: {self.autor})"
37         )
```

- Ejemplo en la clase `Electrónico`

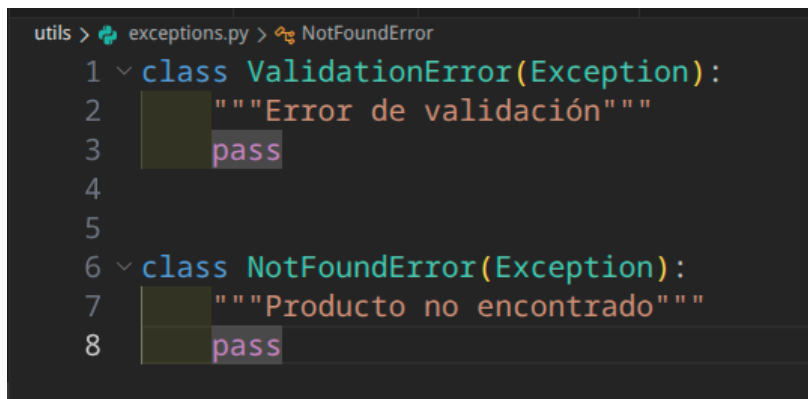
```
models > Electronico.py > Electronico
4 class Electronico(
38 |
39     def calcular_precio_final(self):
40         return self.aplicar_descuento(
41             self.precio,
42             10
43         )
44
45     def __str__(self):
46         return (
47             f"[Electrónico] {self.nombre} | "
48             f"$ / {self.calcular_precio_final():.2f} "
49             f"(Garantía: {self.garantia} meses)"
50         )
```

4. Excepciones: ValidationError, NotFoundError. Bloques try/except/else/finally en flujo de usuario.

Generamos las clases personalizadas de Excepciones

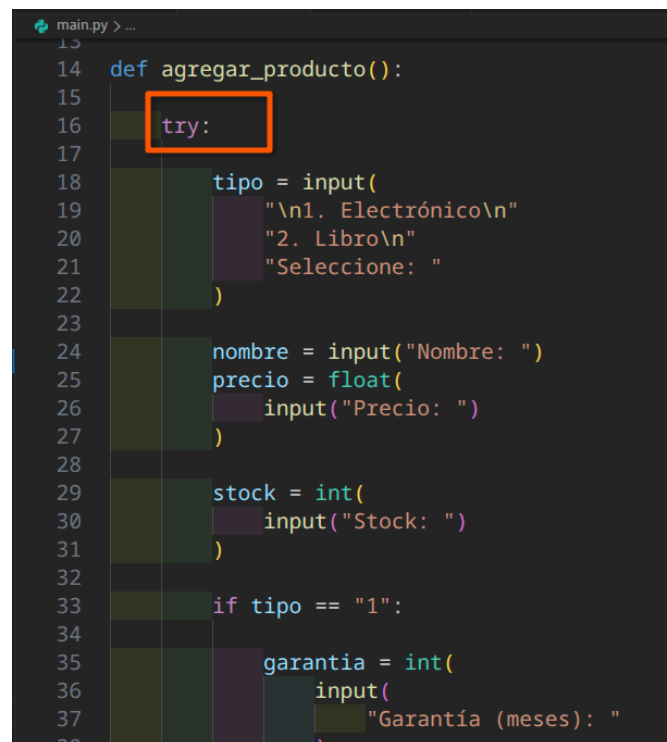


Actualmente las clases se dejan para futuras implementaciones.



El manejo de excepciones se realiza desde el archivo central **main.py**.

- Método **agregar_producto()**:



```

main.py > ...
14 def agregar_producto():
67
68     except ValueError as e:
69         print("ERROR:", e)
70
71     except ValueError:
72         print(
73             "Debe ingresar valores numéricos."
74         )
75
76     else:
77         print(
78             "Producto agregado correctamente."
79         )
80
81     finally:
82         print(
83             "Operación finalizada."
84         )
85

```

- Método `buscar_producto()`:

```

main.py > ...
103 def buscar_producto():
104
105     try:
106
107         nombre = input(
108             "Nombre a buscar: "
109         )
110
111         for producto in productos:
112             if (
113                 producto.nombre.lower()
114                 ==
115                 nombre.lower()
116             ):
117                 print(producto)
118                 return
119
120             raise NotFoundError(
121                 "Producto no encontrado."
122             )
123
124     except NotFoundError as e:
125         print("ERROR:", e)
126

```

5. Modularización: `main.py` (orquestador), `models/` (clases), `utils/` (validaciones, formateo), `__init__.py` en cada paquete.

Estructura de paquetes

```

repo_semana10/
|
├── main.py                ← Orquestador: menú, flujo CLI
|
├── models/
|   ├── __init__.py        ← Exporta todas las clases
|   ├── Producto.py        ← Clase base Producto
|   ├── Electronico.py     ← Clase derivada Electronico
|   ├── Libro.py           ← Clase derivada Libro
|   └── DescuentoMixin.py  ← Mixin de descuento
|
├── utils/
|   ├── __init__.py        ← (inicializador del paquete)
|   ├── exceptions.py      ← Define ValidationError, NotFoundError
|   └── validators.py       ← Define validar_precio(), validar_stock()
|
├── README.md
├── requirements.txt
└── .gitignore

```

models/__init__.py — punto de entrada del paquete

```

__init__.py X
models > __init__.py
1  from .Producto import (
2      Producto
3  )
4
5  from .DescuentoMixin import (
6      DescuentoMixin
7  )
8
9  from .Libro import (
10     Libro
11 )
12
13 from .Electronico import (
14     Electronico
15 )

```

Permite que main.py importe limpiamente con `from models import Electronico` en lugar de `from models.producto import Electronico`.

utils/validators.py — separación de responsabilidades

```

from utils.exceptions import ValidationError

def validar_precio(precio):
    if precio <= 0:
        raise ValidationError(
            "El precio debe ser mayor que cero."
        )

def validar_stock(stock):
    if stock < 0:
        raise ValidationError(
            "El stock no puede ser negativo."
        )

def formatear_moneda(valor):
    return f"S/ {valor:.2f}"

```

Las reglas de validación (precio > 0, stock >= 0) están fuera de las clases, en su propio módulo. Los setters las llaman, pero no las contienen. Esto facilita reutilizarlas y cambiarlas sin tocar las clases.

Capturas del programa funcionando

```

===== MENU =====
1. Agregar
2. Listar
3. Buscar
4. Salir
Opción: 1

1. Electrónico
2. Libro
Seleccione: 1
Nombre: Laptop HP
Precio: 2500
Stock: 10
Garantía (meses): 12
Producto agregado correctamente.
Operación finalizada.

Opción: 2
1. [Electrónico] Laptop HP | S/ 2250.00 (Garantía: 12 meses)

```


Opción: 1

1. Electrónico

2. Libro

Seleccione: 1

Nombre: Mouse

Precio: -50

Stock: 10

Garantía (meses): 6

ERROR: El precio debe ser mayor que cero.

Operación finalizada.

Opción: 3

Nombre a buscar: Tablet

ERROR: Producto no encontrado.